

SW6100 DeepSleep, Hibernate/Quick Boot Feature Overview

80-74841-18 Rev AC

Confidential – Qualcomm Technologies, Inc. and/or its affiliated companies – May Contain Trade Secrets

NO PUBLIC DISCLOSURE PERMITTED: Please report postings of this document on public servers or websites to: DocCtrlAgent@qualcomm.com.

© Qualcomm Technologies, Inc. and/or its subsidiaries. All rights reserved.

Revision History

Revision	Date	Description
AC	March 2026	Updated the following slides with latest information on external companion chip: Slide 7 : <i>DeepSleep</i> Slide 9 : <i>Device Modes On SW6100 Hybrid Wearable Platform</i>
AB	September 2025	Added: Slide 42, <i>Hibernate</i> to Slide 79, <i>OEM Dependencies</i>
AA	July 2025	Initial release

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com

Contents

1 DeepSleep	<u>6</u>
2 Device Modes	<u>8</u>
3 Hardware Architecture	<u>10</u>
4 Software Architecture	<u>13</u>
5 APSS Deep Sleep Entry	<u>15</u>
6 Dependencies	<u>19</u>
7 Non-APSS DeepSleep Entry	<u>21</u>
8 Dependencies	<u>24</u>
9 DeepSleep Trigger from Doze	<u>26</u>
10 Dependencies	<u>29</u>
11 DeepSleep Trigger from User Application	<u>31</u>
12 Dependencies	<u>34</u>
13 Linux Resume from DeepSleep	<u>36</u>
14 Dependencies	<u>40</u>
15 Hibernate	<u>42</u>
16 Architecture	<u>44</u>
17 Functional Requirements	<u>50</u>
18 Dependencies	<u>54</u>
19 Non-APSS Hibernate Entry	<u>56</u>
20 Dependencies	<u>60</u>

Contents

21	Hibernate Trigger from User App	<u>62</u>
22	Dependencies	<u>66</u>
23	Image Restore from Bootloader	<u>68</u>
24	Dependencies	<u>71</u>
25	Linux Resume from Hibernate	<u>73</u>
26	Dependencies	<u>78</u>
27	References	<u>80</u>

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com

Objective

This document describes the SW6100 SoC DeepSleep power state and the software architecture details to enter and exit the state. The SW6100 chipset has introduced two new power modes, in addition to the traditional power modes in the current generation of Qualcomm chipsets, such as, RBSC:

- DeepSleep
- Hibernation or Suspend-to-Disk (S2D)

The objective of this document is to help OEMs who want to improve the battery life of their devices.

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com



Section 1

DeepSleep

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com

DeepSleep

- DeepSleep is a new power state where MSM™ Chipsets digital is entirely turned off (except for pads necessary to exit from this state) and double data rate (DDR) memory is configured for Self-Refresh state.
- Entry to and exit from DeepSleep power state are tightly coordinated across MSM, PMIC, and external companion chip to ensure predictable system behavior.
- As MSM is turned off in DeepSleep, all hardware blocks within the MSM chip lose their state, but application context is saved in DDR.
- The exit from DeepSleep state takes more time than traditional RBSC state, but DeepSleep brings significant power improvements compared to RBSC.
- The following table shows a comparison of the various power states supported on the SW6100 chipset:

	SW6100 MSM	QCC6100	External companion chip (AON)	DDR	Exit latency (to user space)
Hibernate	OFF	OFF	ON	OFF	Tens of seconds
DeepSleep	OFF	OFF	ON	Self-refresh	~800 ms (till kernel resume)*
RBSC	RBSC	OFF	ON	Self-refresh	~100-250 ms
Active	Active	ON	ON	ON	N/A

NOTE: Non-HLOS subsystem resume latency will be additional.



Section 2

Device Modes

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com

Device Modes On SW6100 Hybrid Wearable Platform

Following are the modes available on SW6100 hybrid wearable platform:

■ Active

- This is when the user is interacting with the device. For example, to launch an application or scroll through notifications or a menu.
- The device remains in this state until an inactivity timeout occurs at which point device moves to Tracker mode.

■ Rock bottom sleep (RBSC)

- RBSC is a low power mode. CPU, GPU, and other subsystems are offline, and display is turned off.
- DDR in Self-Refresh mode.

■ DeepSleep

DeepSleep is a new power state where most of the MSM chipsets digital are turned off (except for pads required to exit from this state) and DDR is configured in Self-Refresh mode.

■ Hibernate (S2D)

- The MSM chip is in power off state and the external companion chip remains on to service interrupts and handles minimal tasks.
- HLOS kernel context is saved to storage, which gets restored to DDR post resume.



Section 3

Hardware Architecture

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com

High Level Hardware Architecture

■ PM6100

- Recognizes and supports the DeepSleep power state
- Power grid isolation to support DeepSleep exit logic and DDR SR
- Spare registers to store DeepSleep specific cookies
- Ability to notify DeepSleep entry to companion chip
- Listen for exit events from companion chip
- Event (buttons, real time clock (RTC timer), and so on) routing to companion chip in DeepSleep

■ SW6100 SoC

- Support in MSM to drive the necessary DDR pads to keep DDR in self-refresh
- Support cookies for software to recognize DeepSleep exit

Preconditions To Enter DeepSleep

The system needs to fulfill the following preconditions to enter DeepSleep state:

- Modem should be in Airplane mode
- Display should be off or in Offloaded Ambient mode
- WLAN should not be connected to any AP or turned off
- No sensor algorithms running
- No active GPS tracking
- No active multimedia use cases
- Not be connected to charger or for USB debug

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com

NOTE: More preconditions maybe added in the future depending on the latency of entry and exit to DeepSleep.



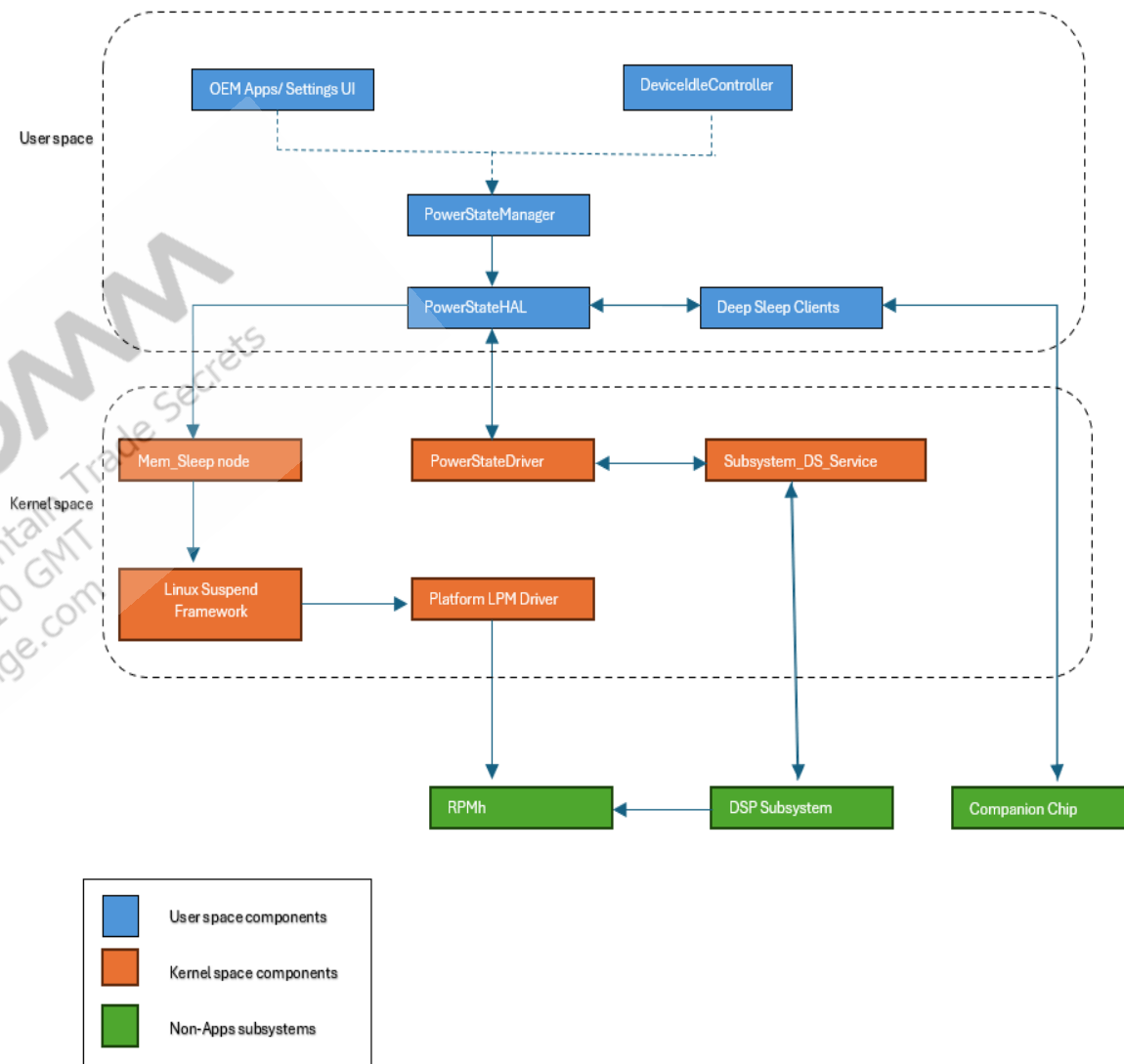
Section 4

Software Architecture

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com

High Level Software Architecture

- Application processor subsystem (APSS) software uses the Wear OS/Linux Suspend-To-RAM framework to enter and resume from DeepSleep.
- System low-power mode (LPM) driver differentiates between DeepSleep and normal system LPM by checking the value of `/sys/power/mem_sleep` node.
- The default value is `s2idle`, which is used for entering MSM RBSC state. The value of `deep` is used by Linux Suspend-to-RAM framework to enter the SoC DeepSleep.
- DeepSleep has higher entry and exit latencies compared to the existing LPMs .
- Following are the triggers to enter DeepSleep
 - Set UI options. (similar to battery saver option).
 - OEMs need to have the correct privilege level to trigger entry to DeepSleep
 - Doze framework





Section 5

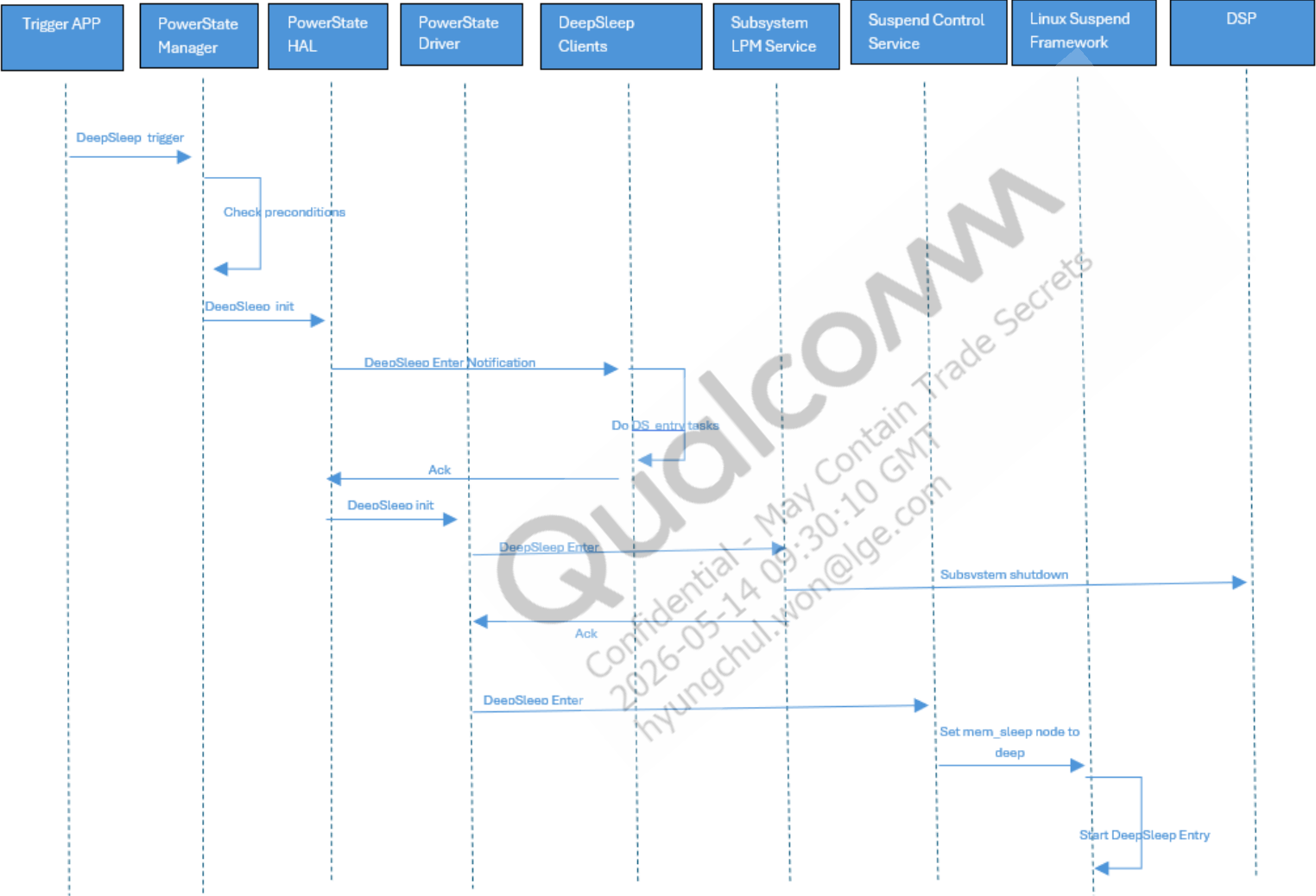
APSS DeepSleep Entry

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyunchul.won@lge.com

APSS DeepSleep Entry

- The APSS handles the complete state machine of entering and resuming from DeepSleep.
- The PowerState HAL in the user space and the PowerState Driver in the kernel implement the state machine and call the registered DeepSleep clients to notify the entry and resume sequence.
- The outcome of the DeepSleep entry state machine is to set `/sys/power/mem_sleep` node to either deep (on success) or shallow (on failure).
- The DeepSleep state machine also depends on Android/Linux Suspend-to-RAM (STR) framework to complete the DeepSleep sequence.
- Once PowerStateHAL sets the `/sys/power/mem_sleep` node to deep, the Linux suspend sequence must be triggered, which happens automatically once all user space and kernel wakelocks/wakeupsources are released.
- The Linux suspend framework iterates through all the drivers that can check the value of `mem_sleep` node to decide the appropriate actions for entering and vote for DeepSleep to resource power manager (RPM).

High Level APSS DeepSleep Entry Sequence



Failure Recovery And Error Codes

DeepSleep entry can fail at multiple levels:

- If the clients return NACK, PowerState HAL gracefully unwinds and resumes notifying the clients that have already executed DeepSleep sequence.
- As DeepSleep depends upon the Linux STR framework, failure to suspend can occur at any point in the suspend sequence. The design gracefully recovers from all of the different types of suspend aborts.
- Once `/sys/power/mem_sleep` node is set to deep and non-APSS have entered DeepSleep, it is expected that Linux suspend starts. However, it could get delayed if any component is holding a wakelock or wake-up source. Check if an appropriate timeout must be added to recover from this state.

Following are the possible error codes associated with recovery failure:

- `DS_ERROR_ENTRY_CRITERIA_FAILURE`
- `DS_ERROR_SUBSYSTEM_FAILURE`
- `DS_ERROR_UNKNOWN`



Section 6

Dependencies

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com

OEM Dependencies

- SuspendControlService is modified to initiate either hibernate or STR, based on inputs from PowerStateService and PowerStateHAL. As a result
 - DeepSleep entry is streamlined as an immediate precursor to the Linux suspend
 - No uncertainty between PowerState Manager executing the DeepSleep sequence
 - Start of Linux suspended
- The user space notifies SuspendControlService that the next entry to suspend is DeepSleep.
- SuspendControlService then takes the following action:
 - Waits for kernel wakelocks to be cleared
 - Informs PowerStateHAL to start the subsystem notification
 - Initiate Linux suspend



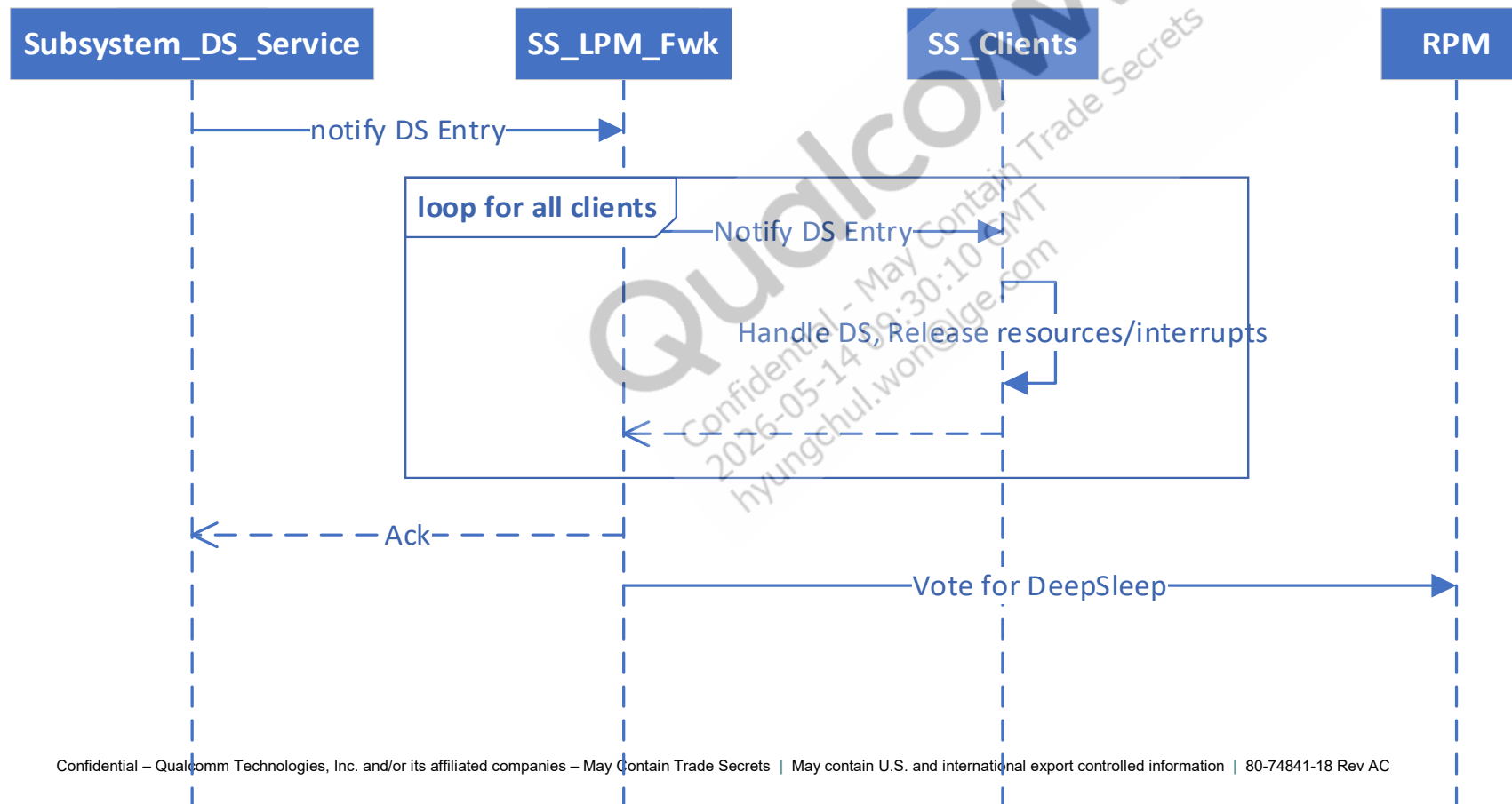
Section 7

Non-APSS DeepSleep Entry

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyunchul.won@lge.com

Non-APSS DeepSleep Entry

- The Subsystem_LPM_Service running on Linux kernel informs modem DSP and low power audio subsystem (LPASS) of the DeepSleep entry . The clients are expected to release the resources being held (for example, clocks, bandwidth votes) and return ACK.
- Once all the clients have released the resources, the LPM framework votes for DeepSleep to the RPM and returns ACK to Subsystem_LPM_Service.
- The following flow chart shows a high level non-APSS DeepSleep entry:



Failure Recovery And Error Codes

- If a subsystem fails to shut down, the SS_LPM_Fwk sends a negative ACK to Subsystem_LPM_Service.
- Subsystem_LPM_Service notifies PowerStateDriver and the entire DeepSleep entry sequence is rolled back.

Following are the possible error codes associated with recovery failure:

- DS_SUCCESS
- DS_ERROR_ENTRY_CRITERIA_FAILURE
- DS_ERROR_SUBSYSTEM_FAILURE
- DS_ERROR_UNKNOWN

NOTE: This includes reversing of all the steps taken till that point (including DeepSleep exit notification to clients, PowerState HAL, PowerState Manager).



Section 8

Dependencies

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com

OEM Dependencies

There are no dependencies on OEMs for this part of the DeepSleep sequence.

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com



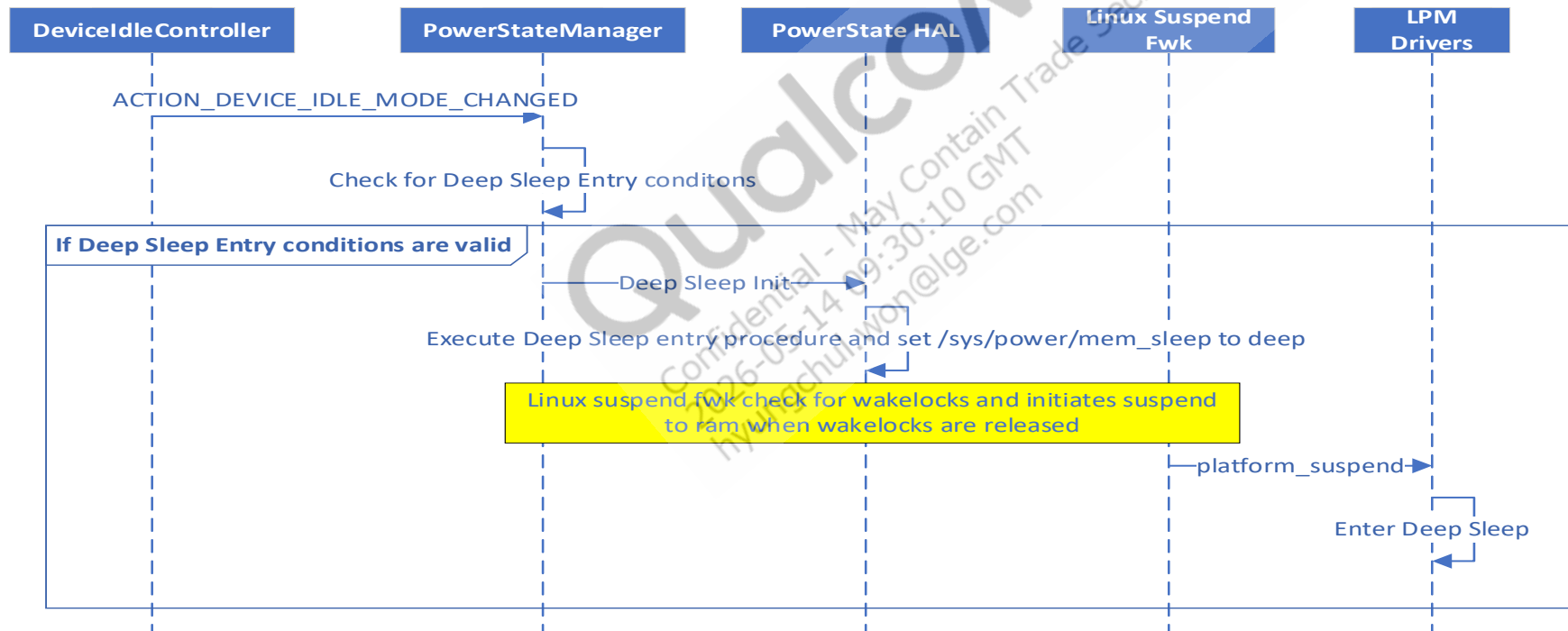
Section 9

DeepSleep Trigger From Doze

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyunchul.won@lge.com

High Level Call Flow – Doze

- Doze or Deep Doze is an Android Low-Power state triggered from user space when device is idle and stationary for certain time (default is 30 minutes).
- In Doze, the network activities and background syncs are deferred to be executed only in the defined maintenance windows.
- As such, DeepSleep can be triggered as part of entry to Deep Doze mode.
- DeviceIdleController handles the entry and exit to Android Light and Deep Doze modes. As part of entry to Doze mode it broadcasts the ACTION_DEVICE_IDLE_MODE intent.
- PowerStateManager can listen to this intent and can start DeepSleep entry if the necessary preconditions, described in [Preconditions to Enter DeepSleep](#), are satisfied.
- The following flow chart shows a high level call flow in Doze mode:



Failure Recovery And Error Codes

Failure to enter DeepSleep can happen if any of the clients do not return ACK to DeepSleep entry call. In this case, the PowerState HAL informs PowerStateManager with appropriate error codes. In failure scenarios, the DeepSleep framework follows the recovery mechanism listed here:

- Subsystems that have already entered DeepSleep is recovered to operational state
- Ensure that HLOS continues shallow system suspend, that is, RBSC

Following are the possible error codes associated with recovery failure:

- DS_SUCCESS
- DS_ERROR_ENTRY_CRITERIA_FAILURE
- DS_ERROR_SUBSYSTEM_FAILURE
- DS_ERROR_UNKNOWN

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com



Section 10

Dependencies

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com

OEM Dependencies

- Add hooks in the DeviceIdleManager (which controls system doze) to inform the PowerStateHAL and trigger DeepSleepSequence
- As DeepSleep is dependent on Android/Linux STR framework, it is ideal if the entry sequence is triggered as part of SuspendControlService (available in current Android versions) initiating suspend from user space. This results in:
 - Streamlined DeepSleep entry as an immediate precursor to the Linux suspend
 - No uncertainty between DeepSleep Manager executing DeepSleep sequence
 - Start Linux suspend.
- The user space notifies SuspendControlService that next entry to suspend can be DeepSleep.
 - SuspendControlService can then wait for kernel wakelocks to be cleared
 - Informs PowerStateHAL to start the subsystem notification
 - Initiate Linux suspend



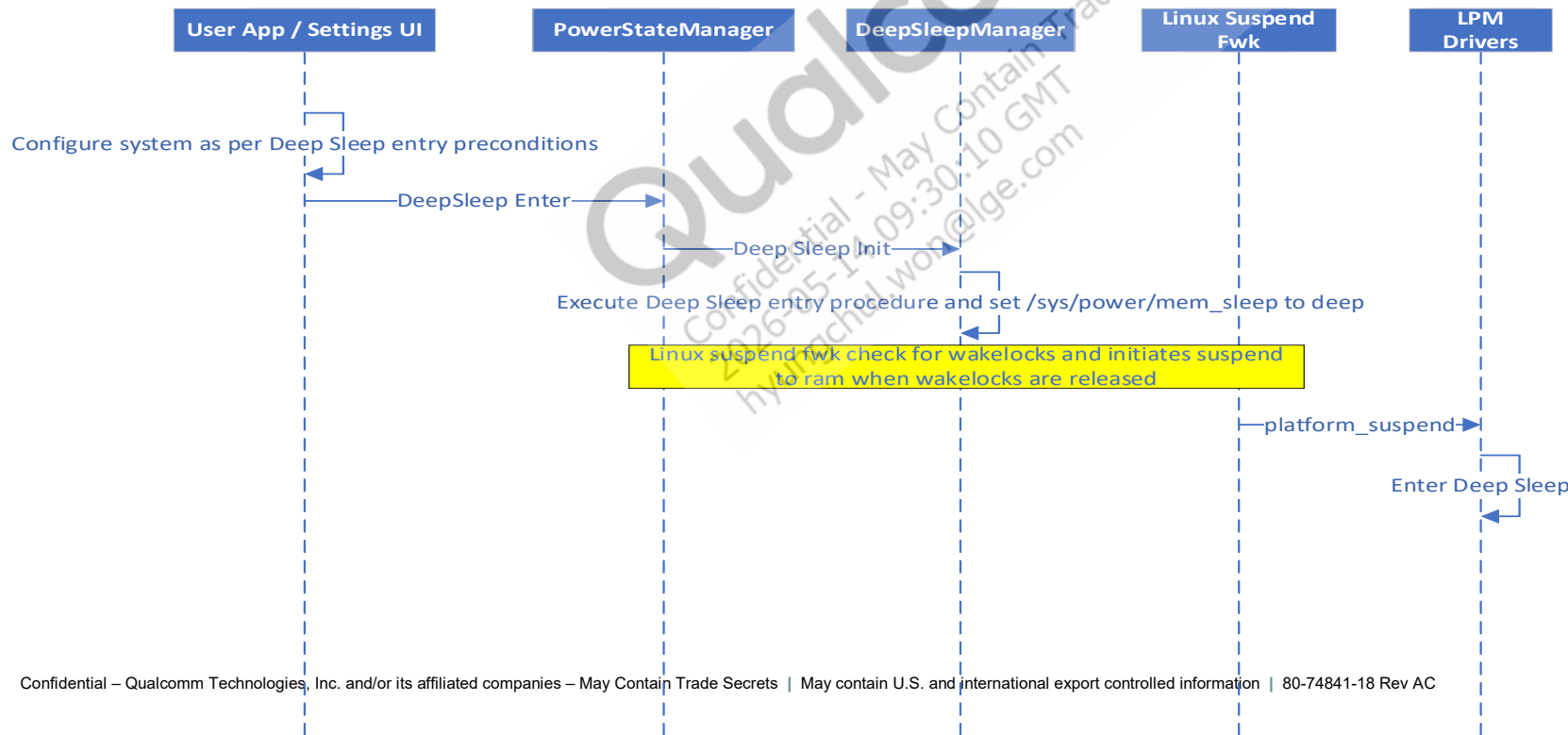
Section 11

DeepSleep Trigger From User Application

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com

DeepSleep Trigger

- DeepSleep can be triggered as part of the settings, for example, when the user selects a Battery Saver mode and so on.
- It is expected that settings and related framework ensure that all the necessary [Preconditions to Enter DeepSleep](#), for the system to enter DeepSleep are configured before calling the PowerStateManager API to trigger DeepSleep entry.
- For example, if the modem is in Connected mode, then the user needs to be notified that modem services will be turned off as part of DeepSleep entry. The modem is put in Airplane mode and appropriate PowerStateManager API is invoked.
- The following flow chart shows a high level call flow when DeepSleep is triggered:



Failure Recovery And Error Codes

Failure to enter DeepSleep can happen if any of the clients do not return ACK to the DeepSleep entry call. The PowerState HAL informs the PowerStateManager with the appropriate error codes. DeepSleep framework provide the following recovery mechanism on a failure scenario:

- Subsystems that have already entered DeepSleep are recovered to operational state
- Ensure that HLOS continues with shallow system suspend, that is, RBSC
- The PowerStateManager returns failure notification to the user application, which inform the user of the error and return the system to pre-DeepSleep entry state

DeepSleep framework returns appropriate error codes to the calling entity. Possible error codes are:

- DS_SUCCESS
- DS_ERROR_ENTRY_CRITERIA_FAILURE
- DS_ERROR_SUBSYSTEM_FAILURE
- DS_ERROR_UNKNOWN



Section 12

Dependencies

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com

OEM Dependencies

- User application needs to be aware of the DeepSleep preconditions and configure the system accordingly. In case of an error, display error prompts the user.
- User space APK needs to ensure that all user space wakelocks are released before triggering the DeepSleep, like suspend is initiated from PowerManagerService

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com



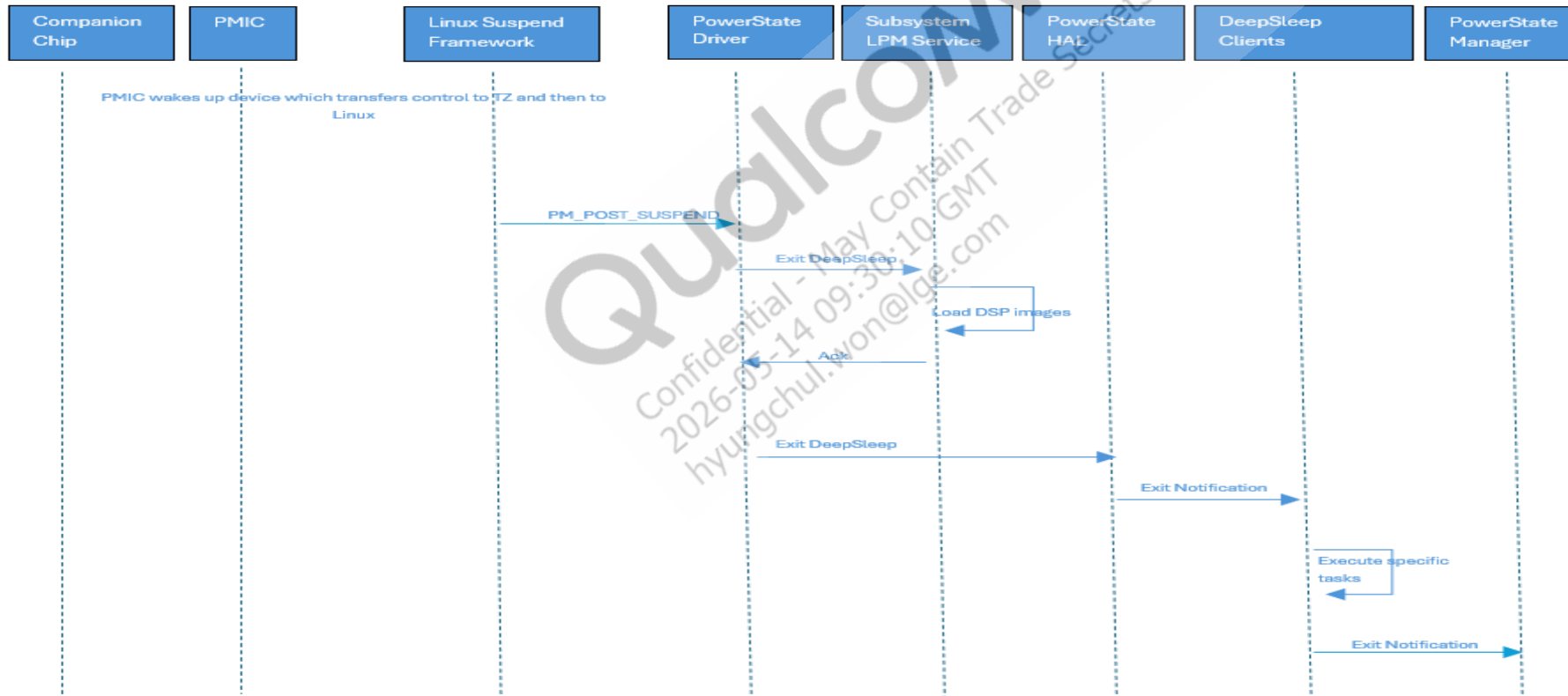
Section 13

Linux Resume From DeepSleep

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com

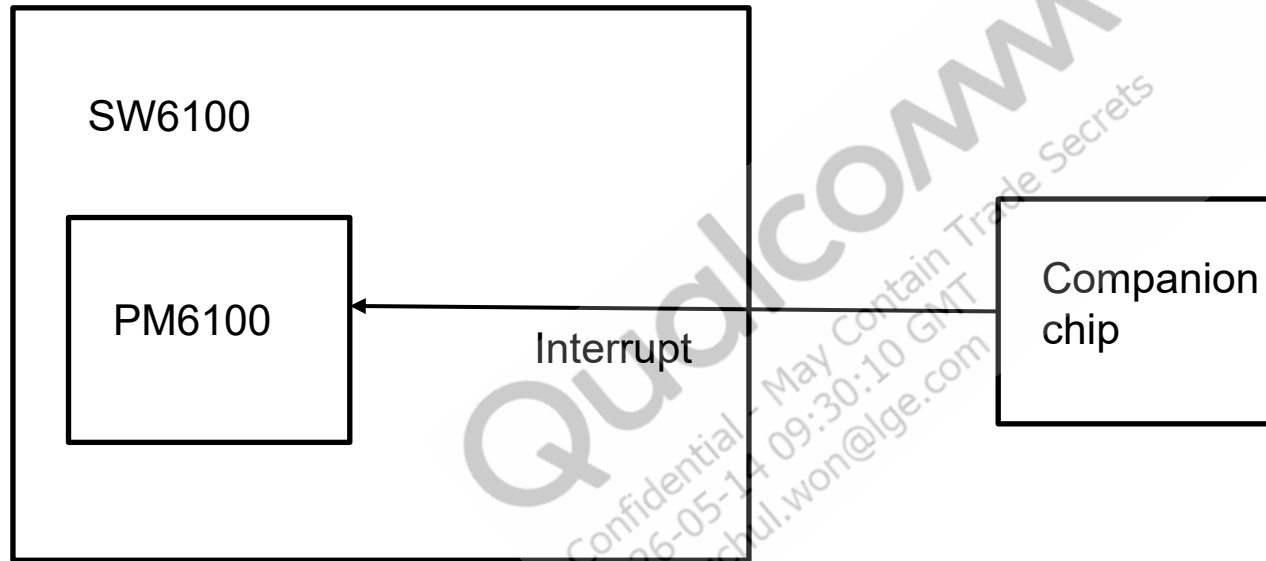
High Level Call Flow For DeepSleep Exit

- DeepSleep exit gets triggered from companion chip, which informs the PMIC and wakes up MSM.
- Companion chip sends a command over I²C to PMIC to trigger DeepSleep.
- PMIC starts the quick boot procedure.
- Once control reaches HLOS, it is like the Linux resume sequence.
- The PowerState HAL receives suspend exit event from Linux and calls `Subsystem_LPM_Service`, which loads Modem DSP and LPASS images through peripheral image loader (PIL).
- The PowerState HAL then informs the registered DeepSleep clients, which perform the exit sequence.



Setup For External Validation Of DeepSleep Exit (Quick Boot)

- A companion chip is required for external validation of DeepSleep exit.
- Upon receiving user input/external event, the companion chip sends interrupt to the PMIC over I²C.



Failure Recovery And Error Codes

- Companion chip failing to interrupt the PMIC for Quick Boot

- Failure in Linux suspend framework/subsystem LPM service

If Linux suspend framework fails to restore the system, or subsystem LPM service fails to restart any subsystem, a system crash is triggered that restarts the system.

- Error in restarting/restoring subsystems/clients

If there is any error in subsystem restart/restoration of user space clients, the recovery depends on the criticality of the error. In case of minor errors, the system runs with a possible impact on functionality. Critical errors, triggers system crash restarting the system.

- Error codes are decided based on the design for failure recovery.

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com



Section 14

Dependencies

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com

OEM Dependencies

Applications or services initiating DeepSleep entry need to be aware or listen to the exit notification to take any specific actions or to inform the user.

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com



Section 15

Hibernate

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com

Hibernate

- Hibernate or Suspend to Disk (S2D) is a power-saving state where most MSM digital components are powered down,
the HLOS kernel image is saved to storage (eMMC/UFS), and DDR memory is completely turned off.
- Hibernate power state entry and exit are tightly coordinated across MSM, PMIC, and Companion Chip to ensure predictable system behavior.
- In the Hibernate state, since both MSM and DDR are powered down, the HLOS kernel context is saved to storage and restored to DDR during resume.
- Exit from the Hibernate state takes longer than the traditional RBSC state.
- Hibernate state brings better power savings than RBSC.



Section 16

Architecture

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com

Hibernate Preconditions

Entering the Hibernate state requires the system to meet specific preconditions:

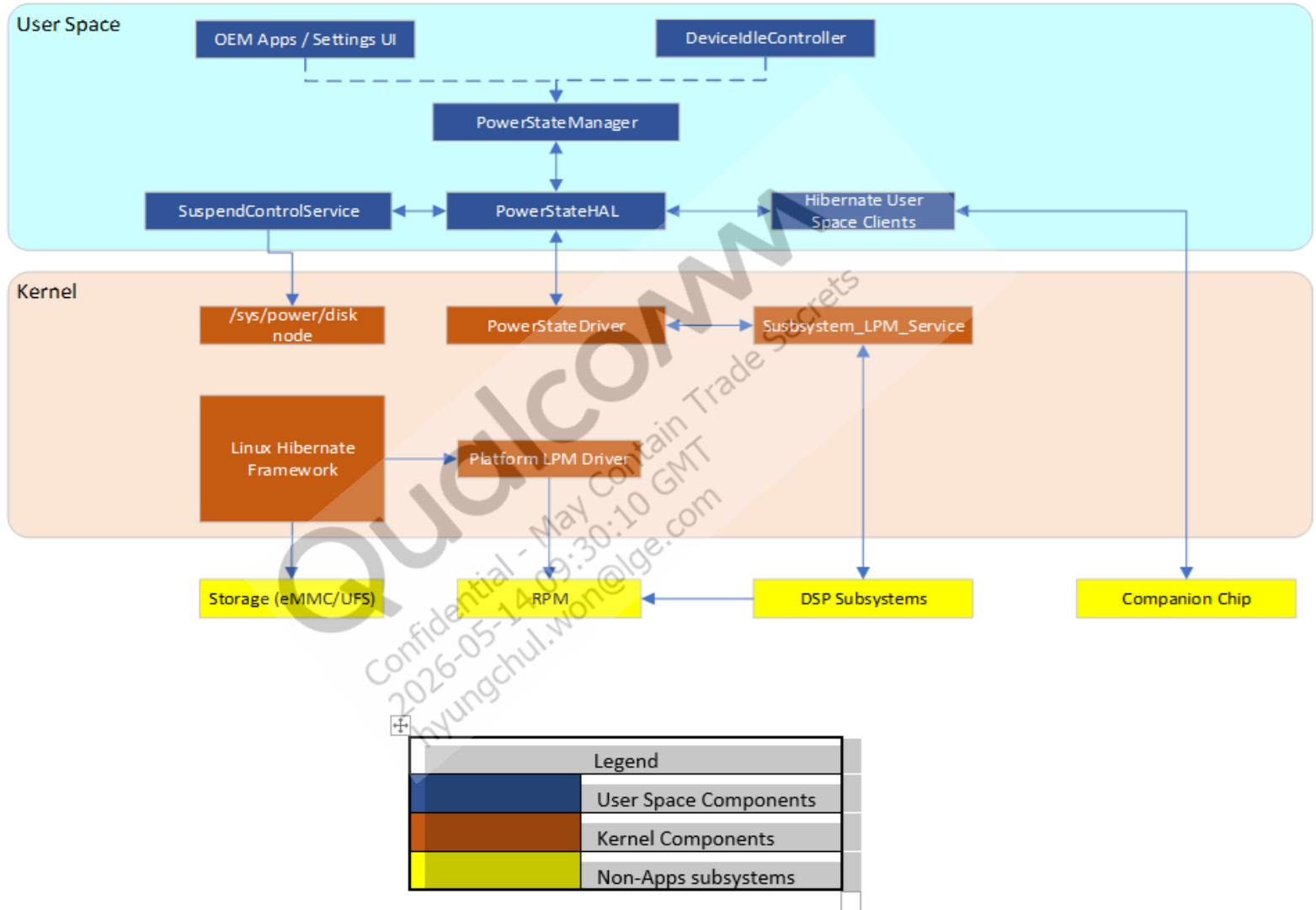
- Modem should be in Airplane mode
- Display should be in Off state or in Offloaded Ambient mode
- WLAN should not be connected to any AP or is turned off
- No sensor algorithm running
- No active GPS tracking
- No active multimedia cases need to be running on the device
- Device should not be connected to charger or for USB debug

NOTE: Preconditions may be added later based on the latency of entry and exit to hibernate and the actual power savings based on device idle time.

High Level Software Architecture

- Hibernate depends on Linux S2D framework.
- The current power management framework on Android/Wear OS (including `PowerManagerService` and `SuspendControlService` does not support S2D.
- To enable Hibernate, modify both `PowerManagerService` and `SuspendControlService`
- These services would need to switch between suspend to RAM and S2D based on inputs from `PowerManagerService` and `SuspendControlService`
- Hibernate mode has higher entry and exit latencies compared to existing LPM. Therefore, it is crucial to define entry conditions to ensure:
 - Longer idle durations are expected
 - The system can tolerate the latency overhead
- The following scenarios can serve as trigger points for entering Hibernate mode
 - Settings UI for Hibernate entry
 - Enable an OEM-specific app with appropriate privileges to trigger Hibernate

Hibernate High Level Software Architecture



Components Of Hibernate Architecture

■ PowerStateManager is a user-space component responsible for:

- Evaluating Hibernate preconditions as defined in the Slide [Hibernate Preconditions](#)
- Triggering the PowerStateHAL to initiate the Hibernate entry sequence once all conditions are met.
- PowerStateManager API can be invoked by:
 - Various entities within the Wear OS framework
 - Application-level code, allowing flexibility in how Hibernate is triggered

■ PowerStateHAL/PowerState Driver

- PowerStateHAL along with PowerStateDriver execute the state machine for Hibernate entry and exit.
- As part of Hibernate entry it sends notification to register Hibernate clients, who in turn execute their specific actions for Hibernate entry and return an acknowledge or failure notification.
- It also calls into `Subsystem_LPM_Service`, which will notify the non-apps subsystems, like modem DSP and LPASS to shut down.
- Upon receiving acknowledge from all clients and `Subsystem_LPM_Service`, PowerStateHAL notifies the SuspendControlService to execute S2D on next suspend entry.

■ Hibernate clients

Hibernate Clients could be any user space entity which wants to take specific action prior to the system enter Hibernate. For example, Sensor HAL could use Hibernate entry/exit events to save sensor data to eMMC. These clients are expected to return success or failure to the PowerStateHAL.

Components of Hibernate Architecture (cont.)

■ Linux Hibernate framework

Hibernate of S2D is executed in Linux kernel in two stages.

- a. User space and drivers are frozen and an image of the RAM is created.
- b. Post this the drivers are thawed to copy the image to storage.
- c. CPUSS is powered off.

■ Subsystem LPM service

Subsystem LPM service communicates with modem DSP and LPASS subsystems to notify them to shut down when entering Hibernate.

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com



Section 17

Functional Requirements

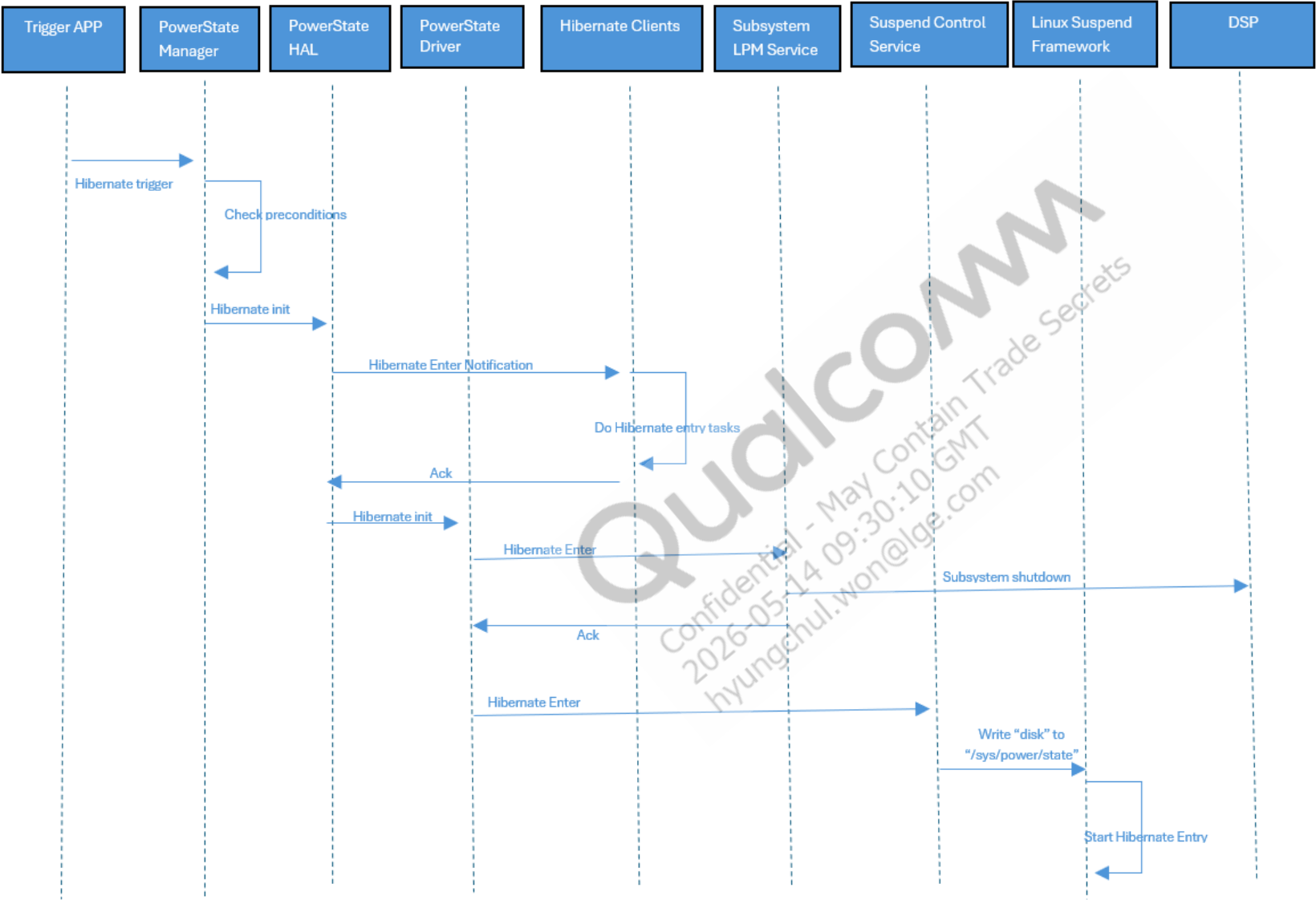
Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyunchul.won@lge.com

APSS Hibernate Entry

Application Processor Subsystem (APSS) handles complete state machine of entering and resuming from Hibernate.

1. PowerStateHAL in the user space implements the state machine and call into the registered Hibernate clients to notify them of the entry and resume sequence.
2. Post notifying the user space clients and shutting down subsystems, PowerStateHAL can notify the SuspendControlService to initiate S2D instead of Suspend-to-RAM on next suspend entry.
3. SuspendControlService chooses the Shutdown Hibernate mode by writing *shutdown* to `/sys/power/disk` and then write disk to `/sys/power/state` to initiate hibernate.
4. The Linux hibernate framework creates the kernel image and saves it to storage before executing the power off sequence.

High Level APSS Hibernate Entry Sequence



Failure Recovery And Error Codes

Hibernate entry can fail at multiple levels:

- If the clients return negative acknowledgement, PowerStateHAL needs to gracefully unwind and resume by notifying clients that Hibernate sequence is already executed, and they can take necessary actions.
- As Hibernate depends upon the Linux S2D framework, failure to suspend can occur at any point of the suspend sequence. The design gracefully recovers from all these different types of suspend aborts.
- Once SuspendControlService is notified to initiate hibernate and non application processor subsystems have been shut down, it is expected that Linux should hibernate.
- The Hibernate entry process may be delayed if any component holds a wakelock or wakeup source, preventing the system from progressing to S2D.
- SuspendControlService should implement an appropriate timeout to allow recovery.
- Possible error codes are
 - HIBERNATE_ERROR_ENTRY_CRITERIA_FAILURE
 - HIBERNATE_ERROR_SUBSYSTEM_FAILURE
 - HIBERNATE_ERROR_UNKNOWN



Section 18

Dependencies

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com

OEM Dependencies

- SuspendControlService needs to be modified to initiate either hibernate or suspend-to-RAM based on inputs from PowerStateService and PowerStateHAL.
- This helps to streamline the Hibernate entry as an immediate precursor to the Linux suspend and removes uncertainty between the completion of the user space Hibernate sequence (handled by PowerStateHAL) and the initiation of Linux's S2D operation.

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com



Section 19

Non-APSS Hibernate Entry

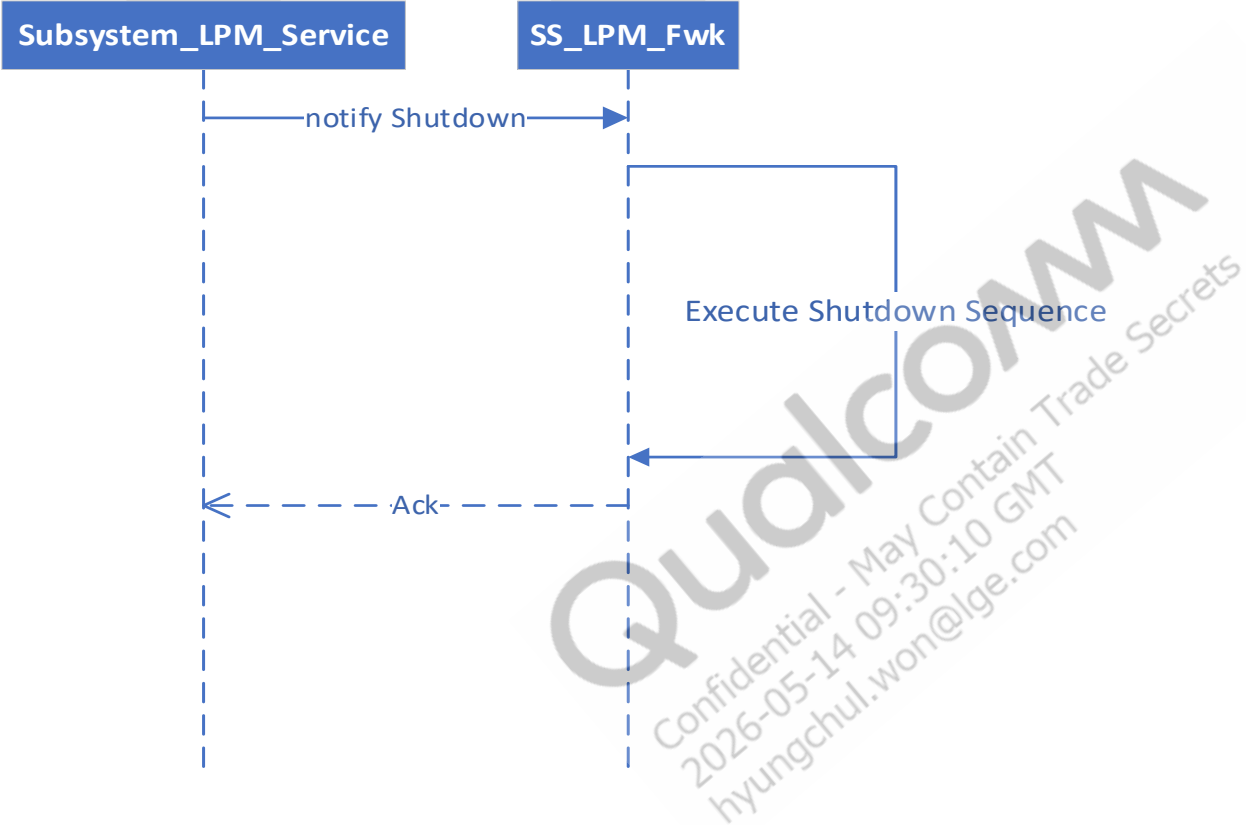
Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com

Non-APSS Hibernate Entry

Modem DSP and LPASS subsystems are notified to shutdown using the `Subsystem_LPM_Service` running on Linux kernel. The clients are expected to release the resources that are held for example, clocks, bandwidth votes before executing shutdown sequence.

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com

High Level Non-APSS Hibernate Entry Sequence



Failure Recovery And Error Codes

- If a subsystem fails to shut down during the Hibernate entry process, the `SS_LPM_Fwk` sends a negative acknowledgment to the `Subsystem_LPM_Service`. Upon receiving this response, the `Subsystem_LPM_Service` notifies the `PowerStateDriver` of the failure.
- As a result, the entire Hibernate entry sequence is rolled back, including hibernation exit notification to clients `PowerState HAL`, `PowerState Manager`.
- The possible error codes are:
 - `HIBERNATE_SUCCESS`
 - `HIBERNATE_ERROR_SUBSYSTEM_FAILURE`
 - `HIBERNATE_ERROR_UNKNOWN`



Section 20

Dependencies

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com

OEM Dependencies

No dependencies on OEM for this part of the Hibernate sequence.

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com



Section 21

Hibernate Trigger From User App

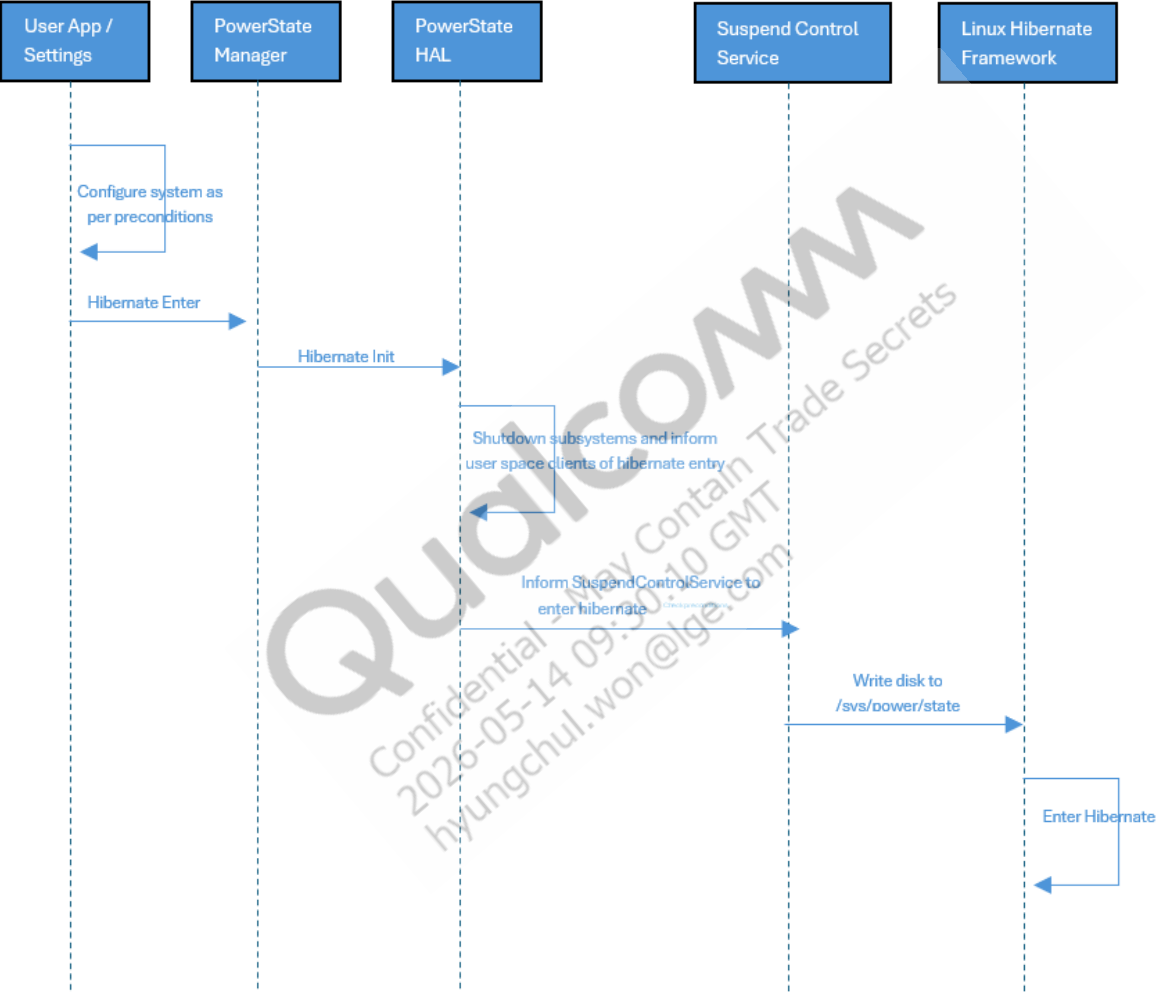
Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com

Hibernate Trigger From User App/Setting UI

- Hibernate can be triggered as part of SettingsUI. When the user selects a battery saver mode it is expected that the SettingsUI and related framework ensure that all the necessary preconditions for the system to enter Hibernate are configured before calling PowerStateManager API.
- For example, if the Modem is in Connected mode, the user must be notified that Modem services will be turned off as part of the Hibernate entry process.
- The SettingsUI puts the modem in Airplane mode and invokes appropriate PowerStateManager API.

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com

High Level Hibernate Entry Sequence From User App



Failure Recovery And Error Codes

- Failure to enter Hibernate can happen if any of the clients do not return an acknowledgement to Hibernate entry call. In this case the PowerStateHAL needs to inform the PowerStateManager with appropriate error codes. In a failure scenario, the following recovery mechanism is provided by the Hibernate framework:
 - Subsystems that have already been shutdown are restored to their operational state.
 - Ensure that HLOS continues with shallow system suspend that is, RBSC
- PowerStateManager returns the failure notification to the user app. The user app informs the user of the error and return the system to pre-Hibernate entry state.
- Hibernate framework returns appropriate error codes to call entity. Following are the possible error codes are:
 - HIBERNATE_SUCCESS
 - HIBERNATE_ERROR_ENTRY_CRITERIA_FAILURE
 - HIBERNATE_ERROR_SUBSYSTEM_FAILURE
 - HIBERNATE_ERROR_UNKNOWN



Section 22

Dependencies

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com

OEM Dependencies

User applications must be aware of the preconditions required for Hibernate and ensure the system is properly configured to meet them. If any errors occur during this process, the app should display appropriate error prompts.

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com



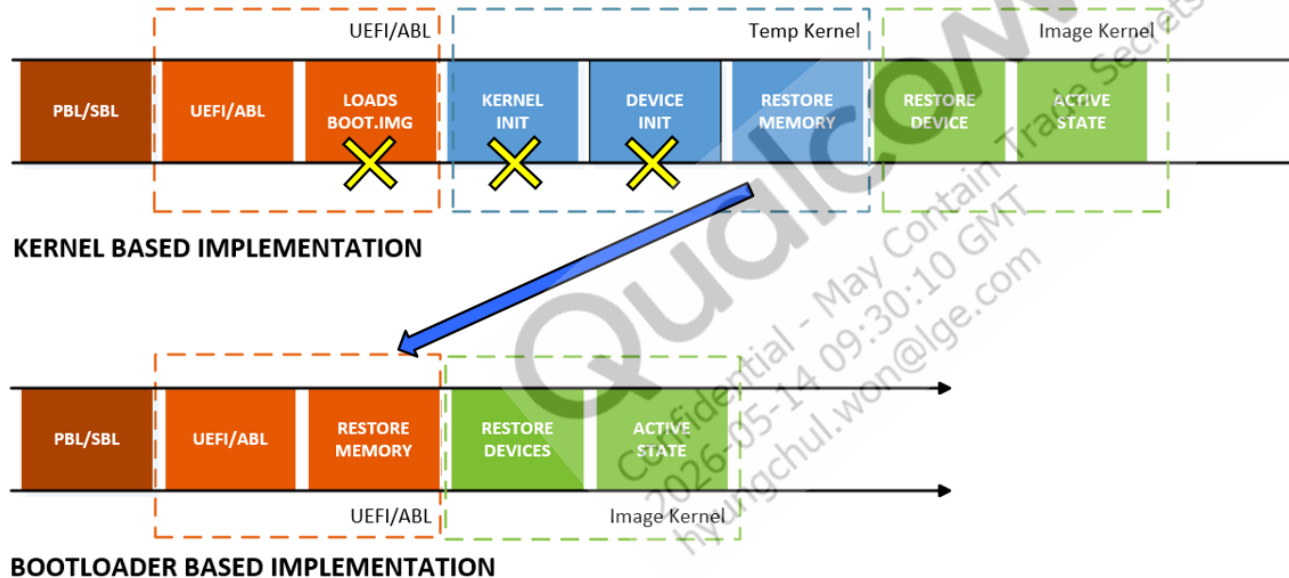
Section 23

Image Restore From Bootloader

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyunchul.won@lge.com

Image Restore From Bootloader

- During resume from Hibernation, the default implementation in the mainline Linux kernel boots a temporary kernel. In the late initialization phase, it checks for a valid hibernation image in the swap disk.
- If no image is found, the system proceeds with a cold boot.
- To improve resume latency, image is restored directly from the bootloader, bypassing the temporary kernel boot.
- Resume from Hibernation starts with cold boot. The Android boot loader (ABL) checks if there is a valid hibernation image in the disk. If a valid hibernation image is detected, it is reloaded to RAM and the system is restored.



Failure Recovery And Error Codes

- If the ABL fails to restore the hibernation image to memory, it aborts the restore sequence and initiates a cold boot.
- As part of this failure, the saved hibernation image is also deleted.
- The previously saved user-space context is lost, and the system boots with a fresh context, just like a normal power-on reset.
- Error codes are decided based on the design for failure recovery.

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com



Section 24

Dependencies

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com

OEM Dependencies

No OEM dependencies identified in this sequence.

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com



Section 25

Linux Resume From Hibernate

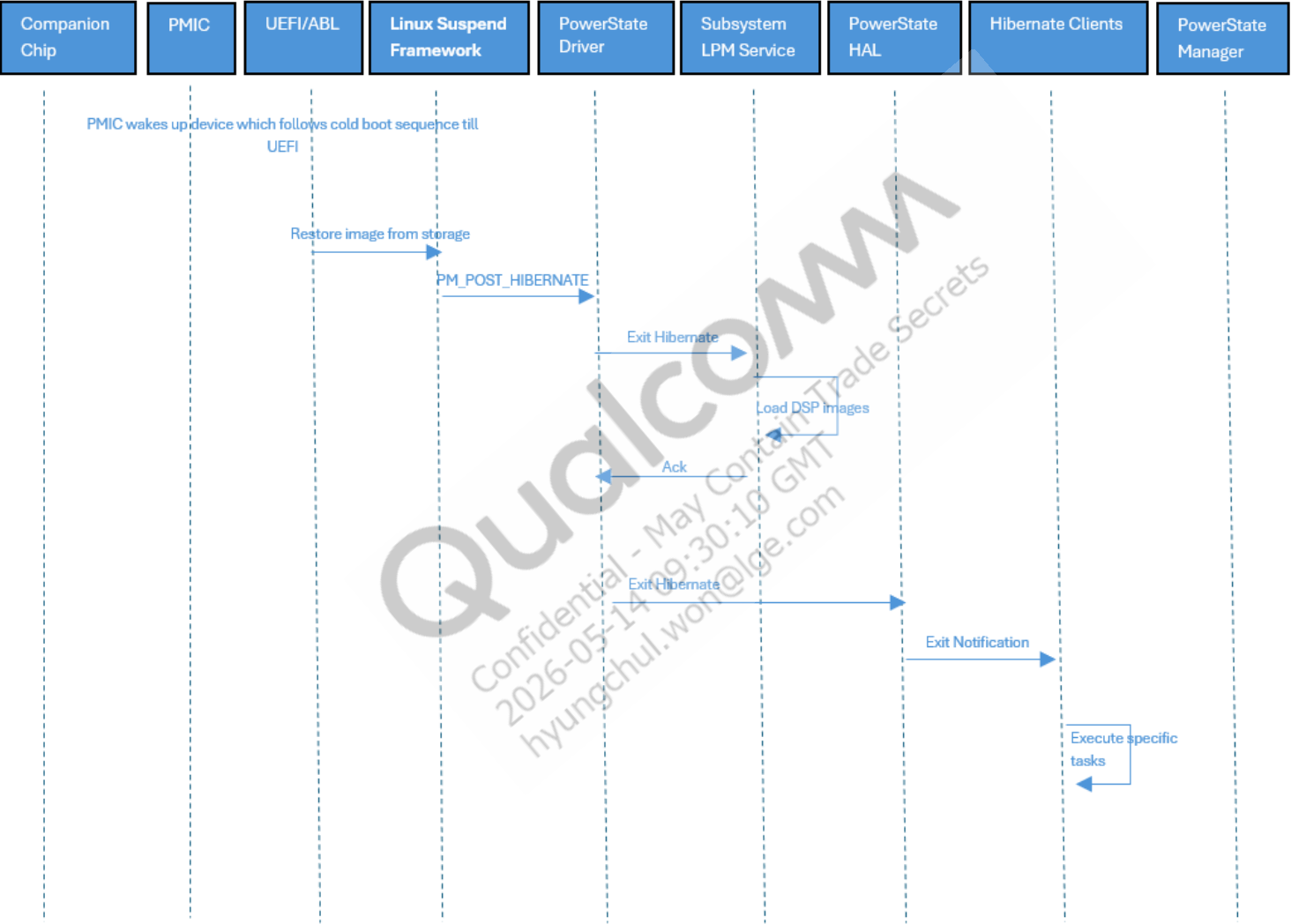
Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyunchul.won@lge.com

Linux Resume From Hibernate

Resume from Hibernate is initiated by the companion chip, which sends a command over I²C to the PMIC. The PMIC then boots up SW6100, triggering the cold boot sequence.

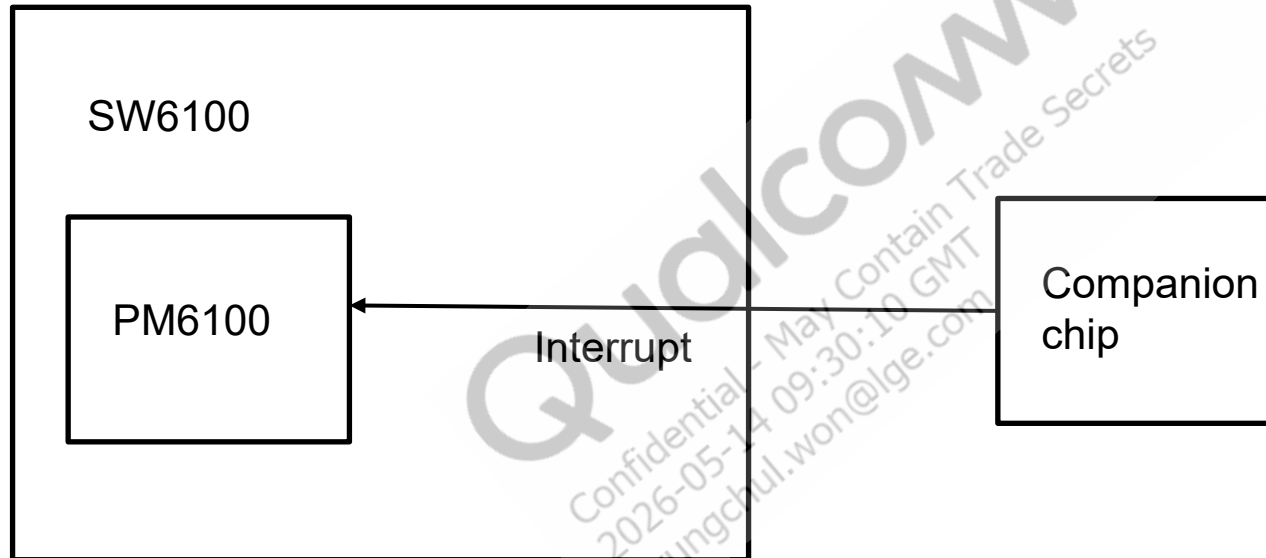
1. The ABL checks the swap disk partition for a valid hibernation image.
2. If a valid image is found, ABL loads it into RAM. It interacts with UEFI to read the disk partition and then transfers control to the hibernation resume entry point. Without a valid image, ABL cannot restore hibernation image.
3. From this point, the Linux kernel takes over the resume process.
4. The PowerStateDriver receives the Hibernation exit event from the kernel and calls into the Subsystem_LPM_Service, which restores the Modem DSP and LPASS images using remoteproc.
5. PowerStateHAL notifies all registered Hibernation clients, allowing them to perform their respective exit sequences and restore user space functionality.

High Level Hibernate Exit Sequence



Setup For External Validation Of Hibernation Exit

To enable external validation and triggering of Hibernate exit, a companion chip is required. Upon detecting a user input or external event, the companion chip sends an interrupt to the PMIC over the I²C interface.



Failure Recovery And Error Codes

The Hibernate restore process can fail at multiple stages. Following are the key failure scenarios:

- The external companion chip may fail to send an interrupt to the PMIC, preventing the system from initiating Hibernate exit.
- Failure in primary bootloader (PBL) or ABL.
- ABL fails to restore the hibernation image.
- If the Linux suspend framework cannot restore the system, or if the `Subsystem_LPM_Service` fails to restart any subsystem, a system crash is triggered, resulting in a full reboot.
- Errors during the restart or restoration of subsystems or user space clients are handled based on severity:
 - Minor error: The system may continue running, possibly with reduced functionality
 - Critical error: A system crash is triggered, leading to a full reboot.
- Error codes are decided based on the design for failure recovery.



Section 26

Dependencies

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com

OEM Dependencies

Applications or services that initiate the Hibernate entry process must listen for Hibernate exit notifications and inform the user.

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com



Section 27

References

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:10 GMT
hyungchul.won@lge.com

Acronyms and Terms

Acronym or Term	Definition
ABL	Android bootloader
APSS	Application processor subsystem
DDR	Double data rate
LPASS	Low-power audio subsystem
LPM	Low_Power mode
PBL	Primary bootloader
PIL	Peripheral image loader
RPM	Resource power manager
S2D	Suspend-to-disk

Questions?

For additional information or to submit technical questions, go to: <https://www.qualcomm.com/support>

LEGAL INFORMATION

Your access to and use of this material, along with any documents, software, specifications, reference board files, drawings, diagnostics and other information contained herein (collectively this “Material”), is subject to your (including the corporation or other legal entity you represent, collectively “You” or “Your”) acceptance of the terms and conditions (“Terms of Use”) set forth below. If You do not agree to these Terms of Use, you may not use this Material and shall immediately destroy any copy thereof.

1) Legal Notice.

This Material is being made available to You solely for Your internal use with those products and service offerings of Qualcomm Technologies, Inc. (“**Qualcomm Technologies**”), its affiliates and/or licensors described in this Material, and shall not be used for any other purposes. If this Material is marked as “**Qualcomm Internal Use Only**”, no license is granted to You herein, and You must immediately (a) destroy or return this Material to Qualcomm Technologies, and (b) report Your receipt of this Material to qualcomm.support@qti.qualcomm.com. This Material may not be altered, edited, or modified in any way without Qualcomm Technologies’ prior written approval, nor may it be used for any machine learning or artificial intelligence development purpose which results, whether directly or indirectly, in the creation or development of an automated device, program, tool, algorithm, process, methodology, product and/or other output. Unauthorized use or disclosure of this Material or the information contained herein is strictly prohibited, and You agree to indemnify Qualcomm Technologies, its affiliates and licensors for any damages or losses suffered by Qualcomm Technologies, its affiliates and/or licensors for any such unauthorized uses or disclosures of this Material, in whole or part.

Qualcomm Technologies, its affiliates and/or licensors retain all rights and ownership in and to this Material. No license to any trademark, patent, copyright, mask work protection right or any other intellectual property right is either granted or implied by this Material or any information disclosed herein, including, but not limited to, any license to make, use, import or sell any product, service or technology offering embodying any of the information in this Material.

THIS MATERIAL IS BEING PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESSED, IMPLIED, STATUTORY OR OTHERWISE. TO THE MAXIMUM EXTENT PERMITTED BY LAW, QUALCOMM TECHNOLOGIES, ITS AFFILIATES AND/OR LICENSORS SPECIFICALLY DISCLAIM ALL WARRANTIES OF TITLE, MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, SATISFACTORY QUALITY, COMPLETENESS OR ACCURACY, AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MOREOVER, NEITHER QUALCOMM TECHNOLOGIES, NOR ANY OF ITS AFFILIATES AND/OR LICENSORS, SHALL BE LIABLE TO YOU OR ANY THIRD PARTY FOR ANY EXPENSES, LOSSES, USE, OR ACTIONS HOWSOEVER INCURRED OR UNDERTAKEN BY YOU IN RELIANCE ON THIS MATERIAL.

Certain product kits, tools and other items referenced in this Material may require You to accept additional terms and conditions before accessing or using those items.

Technical data specified in this Material may be subject to U.S. and other applicable export control laws. Transmission contrary to U.S. and any other applicable law is strictly prohibited.

Nothing in this Material is an offer to sell any of the components or devices referenced herein.

This Material is subject to change without further notification.

In the event of a conflict between these Terms of Use and the *Website Terms of Use* on www.qualcomm.com, the *Qualcomm Privacy Policy* referenced on www.qualcomm.com, or other legal statements or notices found on prior pages of the Material, these Terms of Use will control. In the event of a conflict between these Terms of Use and any other agreement (written or click-through, including, without limitation any non-disclosure agreement) executed by You and Qualcomm Technologies or a Qualcomm Technologies affiliate and/or licensor with respect to Your access to and use of this Material, the other agreement will control.

These Terms of Use shall be governed by and construed and enforced in accordance with the laws of the State of California, excluding the U.N. Convention on International Sale of Goods, without regard to conflict of laws principles. Any dispute, claim or controversy arising out of or relating to these Terms of Use, or the breach or validity hereof, shall be adjudicated only by a court of competent jurisdiction in the county of San Diego, State of California, and You hereby consent to the personal jurisdiction of such courts for that purpose.

2) Trademark and Product Attribution Statements.

Qualcomm is a trademark or registered trademark of Qualcomm Incorporated. Arm is a registered trademark of Arm Limited (or its subsidiaries) in the U.S. and/or elsewhere. The Bluetooth® word mark is a registered trademark owned by Bluetooth SIG, Inc. Other product and brand names referenced in this Material may be trademarks or registered trademarks of their respective owners.

Snapdragon and Qualcomm branded products referenced in this Material are products of Qualcomm Technologies, Inc. and/or its subsidiaries. Qualcomm patented technologies are licensed by Qualcomm Incorporated.